

Caso práctico 1:

El siguiente ejemplo ilustra la primera parte vista de cursores y atributos de cursor: se trata de visualizar los apellidos de los empleados pertenecientes al departamento 20 numerándolos secuencialmente.

```
DECLARE
    CURSOR c1 IS
        SELECT apellido FROM emple WHERE dept_no=20;
    v_apellido VARCHAR2(10);
BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO v_apellido;
        DBMS_OUTPUT.PUT_LINE(TO_CHAR(c1%ROWCOUNT,'99.')||v_apellido);
        EXIT WHEN c1%NOTFOUND;
    END LOOP;
    CLOSE C1;
END;
```

El resultado de la ejecución será:

```
1.SANCHEZ
2.JIMENEZ
3.GIL
4.ALONSO
5.FERNANDEZ
5.FERNANDEZ
```

En este ejemplo observamos que el último FETCH no devuelve ningún valor, no incrementa el atributo %ROWCOUNT y no sobrescribe el valor de la variable del cursor. Es evidente que el programa está mal diseñado, ya que procesa (visualiza) la información supuestamente recuperada antes de comprobar si se ha recuperado información nueva.

En caso de que FETCH no recupere una nueva fila:

- No se incrementará el atributo %ROWCOUNT.
- No se sobrescribe el valor de la variable del cursor.

Práctica propuesta 1.

Escribe el ejercicio anterior subsanando el error de diseño para que no aparezca el último empleado duplicado. Hacerlo primero manteniendo el bucle LOOP...EXIT WHEN, y posteriormente probar con un bucle WHILE. Observa las diferencias en ambos casos.

Caso práctico 2.

En el siguiente ejemplo se visualizan los empleados de un departamento cualquiera usando variables de acoplamiento:

```
CREATE OR REPLACE PROCEDURE ver_emple_por_dept (dep VARCHAR2)
    AS v_dept NUMBER(2);
    CURSOR c1 IS
        SELECT apellido FROM emple WHERE dept_no = v_dept;
    v_apellido VARCHAR2(10);
BEGIN
    v_dept := dep;
    OPEN c1;
    FETCH c1 INTO v_apellido;
    WHILE c1%FOUND LOOP
        DBMS_OUTPUT.PUT_LINE(v_apellido);
        FETCH c1 INTO v_apellido;
    END LOOP;
    CLOSE c1;
END;
```

El programa sustituirá la variable por su valor en el momento en que se abre el cursor, y se seleccionarán las filas según dicho valor. Aunque ese valor cambie durante la recuperación de los datos con FETCH, el conjunto de filas que contiene el cursor no variará.

También podíamos haber usado directamente el parámetro formal *dep* en lugar de la variable *v_dept*. El resultado es el mismo.

Una vez creado el procedimiento, se puede ejecutar:

```
EXECUTE ver_emple_por_dept(30);
```

ARROYO

SALA

MARTIN

NEGRO

TOVAR

JIMENO

Práctica propuesta 2

Escribe un procedimiento que reciba una cadena y visualice el apellido y el número de empleado de todos los empleados cuyo apellido contenga la cadena especificada. Al finalizar, visualiza el número de empleados mostrados. El procedimiento empleará variables de acoplamiento para la selección de filas y los atributos del cursor.

Caso práctico 3.

Escribiremos un bloque PL/SQL que visualice el apellido y la fecha de alta de todos los empleados ordenados por fecha de alta.

1. Mediante una estructura cursor FOR...LOOP.

```
DECLARE
    CURSOR c_emple IS
        SELECT apellido, fecha_alt FROM emple
        ORDER BY fecha_alt;
BEGIN
    FOR v_reg_employ IN c_emple LOOP
        DBMS_OUTPUT.PUT_LINE(v_reg_employ.apellido||'*'||v_reg_employ.fecha_alt);
    END LOOP;
END;
```

2. Utilizando un bucle WHILE.

```
DECLARE
    CURSOR c_emple IS
        SELECT apellido, fecha_alt FROM emple
        ORDER BY fecha_alt;
    v_reg_employ c_emple%ROWTYPE;
BEGIN
    OPEN c_emple;
    FETCH c_emple INTO v_reg_employ;
    WHILE c_emple%FOUND LOOP
        DBMS_OUTPUT.PUT_LINE(v_reg_employ.apellido||'*'||v_reg_employ.fecha_alt);
        FETCH c_emple INTO v_reg_employ;
    END LOOP;
    CLOSE c_emple;
END;
```

Podemos observar que el bucle FOR...LOOP realiza implícitamente la mayoría de las operaciones con el cursor (abrir, comprobar, recuperar fila y cerrar).

Práctica propuesta 3

Escribe el procedimiento realizado anteriormente en la práctica propuesta 2 pero usando un cursor FOR...LOOP. Observa las diferencias con la estructura anterior. Debemos tener en cuenta que el cursor estará cerrado al salir del bucle y no estarán disponibles sus atributos (en concreto %ROWCOUNT).

Caso práctico 4

Escribe un programa que muestre, en formato similar a las rupturas de control o secuencia los siguientes datos:

- Para cada empleado: apellido y salario.
- Para cada departamento: número de empleados y suma de los salarios del departamento.
- Al final del listado: número total de empleados y suma de todos los salarios.

```

CREATE OR REPLACE PROCEDURE listar_emple
AS
CURSOR c1 IS
    SELECT apellido, salario, dept_no FROM emple
        ORDER BY dept_no, apellido;
vr_emp c1%ROWTYPE;
dep_ant EMPL.DEPT_NO%TYPE DEFAULT 0;
cont_emple NUMBER(4) DEFAULT 0;
sum_sal NUMBER(9,2) DEFAULT 0;
tot_emple NUMBER(4) DEFAULT 0;
tot_sal NUMBER(10,2) DEFAULT 0;
BEGIN
OPEN c1;
LOOP
    FETCH c1 INTO vr_emp;
    /* Si es el primer Fetch inicializamos dep_ant */
    IF c1%ROWCOUNT = 1 THEN
        dep_ant := vr_emp.dept_no;
    END IF;
    /* Comprobación nuevo departamento (o finalización) y resumen
del anterior e inicialización de contadores y acumuladores parciales
*/
    IF dep_ant <> vr_emp.dept_no OR c1%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('*** DEPTO: ' || dep_ant ||
            ' NUM. EMPLEADOS: ' || cont_emple ||
            ' SUM. SALARIOS: ' || sum_sal);
        dep_ant := vr_emp.dept_no;
        tot_emple := tot_emple + cont_emple;
        tot_sal := tot_sal + sum_sal;
        cont_emple := 0;
        sum_sal := 0;
    END IF;
    EXIT WHEN c1%NOTFOUND;
    /* Condición de salida del bucle */
    /* Escribir Líneas de detalle incrementar y acumular */
    DBMS_OUTPUT.PUT_LINE(RPAD(vr_emp.apellido,10) || ' * '
        ||LPAD(TO_CHAR(vr_emp.salario,'999,999'),12));
    cont_emple := cont_emple + 1;
    sum_sal := sum_sal + vr_emp.salario;
END LOOP;

```

```

CLOSE c1;
/* Escribir totales informe */
DBMS_OUTPUT.PUT_LINE(' ***** NUMERO TOTAL EMPLEADOS: '
                      || tot_emple || ' TOTAL SALARIOS: ' || tot_sal);
END listar_emple;

```

Práctica propuesta 4

Haz los cambios necesarios en el programa anterior para que realice el mismo listado usando una estructura de CURSOR FOR...LOOP (hay que tener en cuenta el ámbito de las variables de registro del cursor).

Hecho esto, podemos incluir en el programa rupturas por oficio, indicando en este caso únicamente el nombre del oficio y el número de empleados que tiene. Se entiende que se mantienen las rupturas por departamento y los subtotales; y dentro de cada departamento se harán, rupturas por oficio.

Caso práctico 5

El siguiente ejemplo recibe un número de empleado y una cantidad que se incrementará al salario del empleado correspondiente. Utilizaremos dos excepciones, una definida por el usuario salario_nulo y la otra predefinida NO_DATA_FOUND.

```

CREATE OR REPLACE PROCEDURE subir_salario( num_empleado INTEGER, incremento REAL)
IS
    salario_actual REAL;
    salario_nulo EXCEPTION;
BEGIN
    SELECT salario INTO salario_actual FROM emple
        WHERE emp_no = num_empleado;
    IF salario_actual IS NULL THEN
        RAISE salario_nulo; -- levanta salario_nulo
    END IF;
    UPDATE emple SET salario := salario + incremento
        WHERE emp_no = num_empleado;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE(num_empleado||'*Err.No encontrado');
    WHEN salario_nulo THEN
        DBMS_OUTPUT.PUT_LINE(num_empleado||'*Err. Salario nulo');
END subir_salario;

```

Caso práctico 6

El siguiente ejemplo ilustra lo estudiado hasta ahora respecto a la gestión de excepciones. Crearemos un bloque donde se define la excepción `err_blanco` asociada con un error definido por el programador y la excepción `no_hay_espacio` asociándola con el error número -1547 de Oracle.

Tablas usadas para esta práctica guiada:

Desc TEMP2;

Nombre	¿Nulo?	Tipo
COL1	NOT NULL	VARCHAR2(20)
COL2		VARCHAR2(20)

Desc CLIENTESS;

Nombre	¿Nulo?	Tipo
NIF	NOT NULL	VARCHAR2(10)
NOMBRE		VARCHAR2(15)

DECLARE

```

cod_err number(6);
vnif varchar2(10);
vnom varchar2(15);
err_blanco EXCEPTION;
no_hay_espacio EXCEPTION;
PRAGMA EXCEPTION_INIT(no_hay_espacio, -1547);

```

BEGIN

```

SELECT col1, col2 INTO vnif, vnom FROM TEMP2;
IF SUBSTR(vnom,1,1) <= ' ' THEN
    RAISE err_blanco;
END IF;
UPDATE clientes SET nombre = vnom WHERE nif = vnif;
EXCEPTION
    WHEN err_blanco THEN
        INSERT INTO temp2(col1) VALUES ('ERR blanco');
    WHEN no_hay_espacio THEN
        INSERT INTO temp2(col1) VALUES ('ERR tablespace');
    WHEN NO_DATA_FOUND THEN
        INSERT INTO temp2(col1) VALUES ('ERR no habia datos');
    WHEN TOO_MANY_ROWS THEN
        INSERT INTO temp2(col1) VALUES ('ERdemasiados datos');
    WHEN OTHERS THEN
        cod_err := SQLCODE;
        INSERT INTO temp2(col1) VALUES (cod_err);

```

END;

Respecto a las excepciones que aparecen:

- error_blanco hay que declararla y levantarla.
- no_hay_espacio hay que declararla y asociarla con el error de Oracle, pero no hay que levantarla.
- NO_DATA_FOUND y TOO_MANY_ROWS no hay que declararlas ni asociarlas. Tampoco hay que levantarlas.
- El manejador WHEN OTHERS cazará cualquier otra excepción e insertará el código de error de Oracle en la tabla temp2 que suponemos creada.